Classification de Races de Chiens

par Machines à Vecteurs de Support (SVM)

Rapport de Projet – Module Machine Learning

---

11 juin 2026

Résumé

Ce rapport présente la conception et l'évaluation d'un pipeline de classification automatique de races de chiens à partir d'images, en utilisant des machines à vecteurs de support (SVM). Nous décrivons successivement la base de données utilisée, les méthodes d'extraction de caractéristiques (HOG, couleur, PCA par classe), les stratégies multiclasses OvO et OvR, puis une comparaison des approches développées par les deux membres de l'équipe.

Table des matières

1 Base de données **2**
1.1 Description générale . . . 2
1.2 Chargement et rognage des images . . . 2
1.3 Distribution des classes . . . 2
1.4 Quantification de la similarité train/test . . . 3

2 Création des features **4**
2.1 Philosophie générale : le modèle « Lego » . . . 4
2.2 Descripteur de forme : HOG (Histogram of Oriented Gradients) . . . 4
2.2.1 Calcul des gradients par l'opérateur de Sobel . . . 4
2.2.2 Magnitude et orientation du gradient . . . 4
2.2.3 Accumulation spatiale : la construction des histogrammes . . . 5
2.2.4 Implémentation RGB . . . 5
2.2.5 Implémentation HSV-HOG . . . 6
2.3 Réduction dimensionnelle : PCA par classe . . . 7
2.4 Calcul de la PCA : Deux approches mathématiques . . . 7
2.5 Analyse de la variance cumulée et sélection des composantes . . . 8
2.6 Pipeline complet . . . 9
2.7 Optimisation des hyperparamètres (GridSearchCV) . . . 10

3 OvO vs OvR **10**
3.1 Contexte : le problème multiclasse des SVM . . . 10
3.1.1 One-vs-One (OvO) . . . 10
3.1.2 One-vs-Rest (OvR) . . . 10
3.2 Comparaison des temps d'entraînement . . . 11
3.3 Matrices de confusion . . . 11
3.4 Analyse des erreurs . . . 12
3.5 Discussion . . . 12

4 Application à la Reconnaissance Faciale **13**
4.1 Protocole expérimental et base de données . . . 13
4.2 Performances de classification et analyse de la séparabilité . . . 13
4.3 Reconstruction d'image et approximation spectrale . . . 14
4.4 Génération de visages synthétiques et visage chimère . . . 16

Conclusion **16**

1 Base de données

1.1 Description générale

La base de données utilisée, `SmallDB`, est un sous-ensemble de la base Stanford Dogs Dataset. Elle contient des images de six races de chiens : **Chihuahua**, **Basset**, **Kerry Blue Terrier**, **Groenendael**, **Malinois** et **Chow**. Chaque image est accompagnée d'un fichier d'annotation XML au format Pascal VOC indiquant la *bounding box* délimitant le chien dans l'image.

1.2 Chargement et rognage des images

La première étape du prétraitement consiste à lire les images et à les recadrer (*crop*) autour du sujet d'intérêt grâce aux annotations XML. La fonction `extract_bounding_box` analyse les fichiers XML pour récupérer les coordonnées $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$, puis `read_and_crop` extrait la région d'intérêt correspondante.

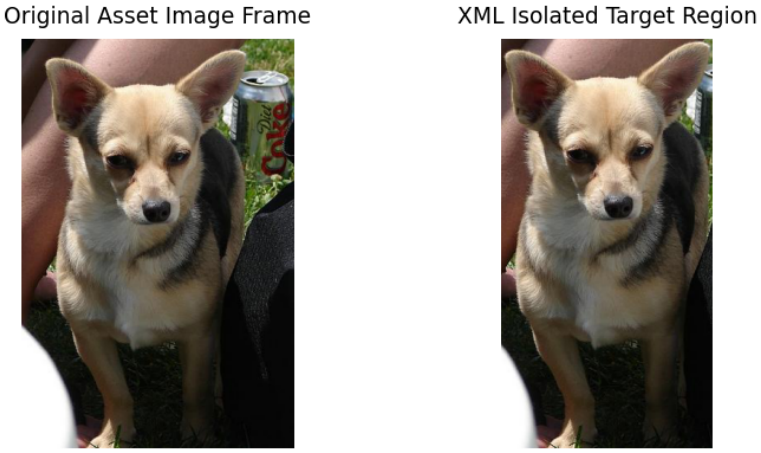


FIGURE 1 – Exemple de rognage XML : image originale (gauche) et région isolée (droite) pour un Chihuahua. La bounding box Pascal VOC permet de supprimer l'arrière-plan non informatif.

1.3 Distribution des classes

Une fois les images chargées et rognées, la base est divisée en un ensemble d'entraînement (75 %) et un ensemble de test (25 %) par un découpage stratifié (`stratify=labels`, `random_state=42`). La stratification garantit que les proportions par race sont préservées dans les deux partitions.

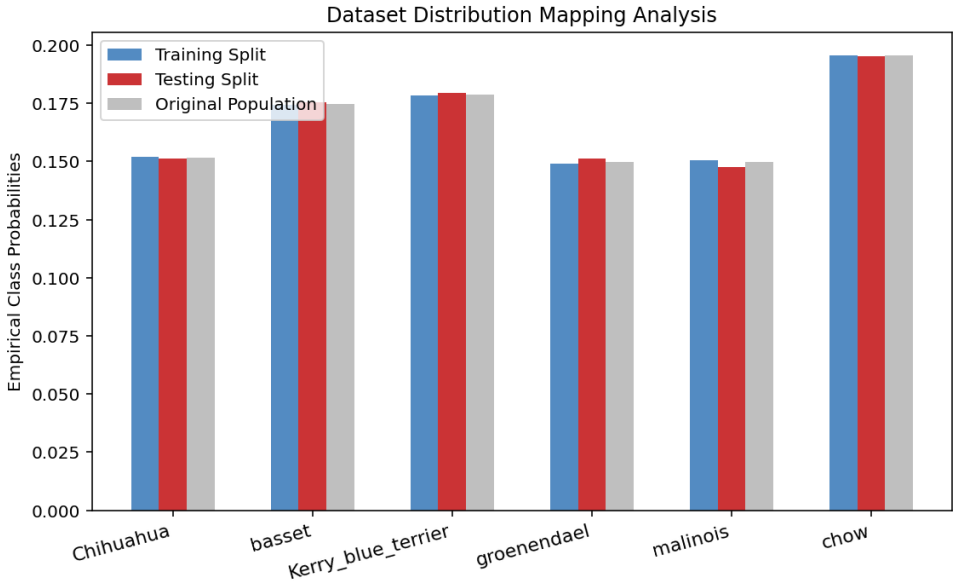


FIGURE 2 – Analyse de la distribution des classes (Dataset Distribution Mapping Analysis). Les barres bleues, rouges et grises représentent respectivement les proportions dans l'ensemble d'entraînement, de test et la population globale. Les proportions sont quasi-identiques grâce au découpage stratifié.

1.4 Quantification de la similarité train/test

Pour s'assurer de l'absence d'un *« effet machine à remonter le temps »* (fuite de données temporelle), la ressemblance entre les distributions empiriques des deux ensembles est mesurée par la **similarité cosinus** :

$$\text{Similarité Cosinus}(\mathbf{p}_{\text{train}}, \mathbf{p}_{\text{test}}) = \frac{\mathbf{p}_{\text{train}} \cdot \mathbf{p}_{\text{test}}}{\|\mathbf{p}_{\text{train}}\| \cdot \|\mathbf{p}_{\text{test}}\|} \tag{1}$$

où $\mathbf{p}$ désigne le vecteur des probabilités empiriques par classe. Une valeur proche de 1 indique que les deux distributions sont très semblables.

```python
def emp_dist(y_train, y_test):
    _, occ_train = np.unique(y_train, return_counts=True)
    _, occ_test  = np.unique(y_test,  return_counts=True)
    prop_train = occ_train / sum(occ_train)
    prop_test  = occ_test  / sum(occ_test)
    return prop_train, prop_test

def cos_sim(prop_train, prop_test):
    return np.dot(prop_train, prop_test) / (
        np.linalg.norm(prop_train) * np.linalg.norm(prop_test)
    )
```

Listing 1 – Calcul des distributions empiriques et de la similarité cosinus.

Les deux équipes ont obtenu une similarité cosinus très proche de 1, confirmant la bonne homogénéité du découpage stratifié.

2 Création des features

2.1 Philosophie générale : le modèle « Lego »

Les caractéristiques sont encapsulées dans des transformateurs scikit-learn (héritage de `BaseEstimator` et `TransformerMixin`). Cette architecture permet leur composition modulaire dans un `Pipeline` et garantit l'absence de fuite de données : toutes les étapes d'estimation (PCA, scalers) sont entraînées *uniquement* sur les données d'entraînement.

2.2 Descripteur de forme : HOG (Histogram of Oriented Gradients)

Le descripteur HOG repose sur l'hypothèse que l'apparence et la forme locales d'un objet peuvent être décrites de manière optimale par la distribution des intensités de gradients de directions. Le pipeline de calcul de ce descripteur se décompose en trois phases majeures : le calcul des gradients, le masquage spatial en cellules (binning) et la normalisation par bloc.

2.2.1 Calcul des gradients par l'opérateur de Sobel

Pour capturer les contours et les structures fines des images de chiens ou de visages, nous appliquons l'opérateur de Sobel. Le filtre Sobel permet d'approcher le gradient tout en gardant un lissage gaussien pour limiter l'impacte du bruit dans les calcul.

Soit I la matrice d'intensité lumineuse d'un canal de l'image. Les approximations des dérivées partielles horizontales (G_h) et verticales (G_v) sont obtenues par la convolution de l'image I avec les noyaux de Sobel K_h et K_v de dimension 3×3 :

$$K_h = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_v = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} \tag{2}$$

Pour chaque pixel de coordonnées (i, j), les composantes du gradient sont calculées par application locale de ces masques :

$$G_h(i,j) = (I * K_h)(i,j) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} K_h(m+1, n+1) \cdot I(i-m, j-n) \tag{3}$$

$$G_v(i,j) = (I * K_v)(i,j) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} K_v(m+1, n+1) \cdot I(i-m, j-n) \tag{4}$$

2.2.2 Magnitude et orientation du gradient

Une fois les cartes de gradients de directions G_h et G_v établies, nous convertissons ces coordonnées cartésiennes en coordonnées polaires pour obtenir la magnitude $m(i, j)$ et l'orientation $\theta(i, j)$ associées à chaque pixel :

$$m(i,j) = \sqrt{G_h(i,j)^2 + G_v(i,j)^2} \tag{5}$$

L'orientation de base est calculée à l'aide de la fonction arc-tangente à deux arguments (`np.arctan2` dans l'implémentation python), permettant de lever toute ambiguïté sur le quadrant géométrique :

$$\theta_{signed}(i,j) = \arctan2\left(G_v(i,j), G_h(i,j)\right) \in [-\pi, \pi] \tag{6}$$

Dans le cadre de la reconnaissance d'objets ou de formes physiques stables (comme la truffe ou les oreilles d'un chien), les gradients dits **non signés** s'avèrent plus performants car ils ignorent le sens de transition du contraste (clair-vers-sombre ou sombre-vers-clair). Nous appliquons donc une projection pour contraindre l'espace des angles sur l'intervalle $[0, \pi[$:

$$\theta(i,j) = \theta_{signed}(i,j) \pmod{\pi} \tag{7}$$

Concrètement, dans le script, cette opération se traduit par l'instruction conditionnelle vectorisée : `orientation[orientation < 0] += np.pi`.

2.2.3 Accumulation spatiale : la construction des histogrammes

L'image est ensuite segmentée en une grille de cellules de taille uniforme ($N_h \times N_w$). Pour chaque cellule, un histogramme local à B classes (*bins*) est initialisé. Chaque pixel de la cellule apporte sa contribution (son « vote ») au canal d'orientation correspondant à sa valeur $\theta(i,j)$.

Le poids du vote n'est pas unitaire : il est directement proportionnel à la magnitude $m(i,j)$ du pixel. Ainsi, un pixel situé sur un contour net et prononcé influencera fortement l'histogramme, tandis qu'un pixel d'une zone plane et homogène (faible magnitude) aura un impact négligeable. Le vecteur final du descripteur HOG est issu de la concaténation ordonnée de tous les histogrammes de cellules.

2.2.4 Implémentation RGB

Nous utilisons une variante couleur (`extraire_caracteristiques_couleur`) : pour chaque cellule, un histogramme est calculé séparément sur chacun des trois canaux R, G, B. La grille est 4×4 avec $B = 9$ bins, donnant un vecteur de taille $4 \times 4 \times 9 \times 3 = 432$. On concatène la matrice de rendu avec la matrice HOG de l'image en niveaux de gris, en appliquant la pondération suivante trouvée sur Internet. $img_grise = 0.299 * img_color_resized[:,:,0] + 0.587 * img_color_resized[:,:,1] + 0.114 * img_color_resized[:,:,2]$.

```
def extraire_caracteristiques_couleur(image, nb_height_cells=4,
    nb_width_cells=4, nb_bins=9):
     [...]
     for i in range(nb_height_cells):
         for j in range(nb_width_cells):
             for k in range(cell_h):
                 for l in range(cell_w):
```

```
                    bin_index_r = int(image[i*cell_h + k, j*cell_w +
    l, 0] // bin_width)
                    bin_index_g = int(image[i*cell_h + k, j*cell_w +
    l, 1] // bin_width)
                    bin_index_b = int(image[i*cell_h + k, j*cell_w +
    l, 2] // bin_width)
                    if bin_index_r == nb_bins:
                        bin_index_r -= 1
                    if bin_index_g == nb_bins:
                        bin_index_g -= 1
                    if bin_index_b == nb_bins:
                        bin_index_b -= 1
                    output[i, j, bin_index_r,0] += image[i*cell_h + k
    , j*cell_w + l, 0]
                    output[i, j, bin_index_g,1] += image[i*cell_h + k
    , j*cell_w + l, 1]
                    output[i, j, bin_index_b,2] += image[i*cell_h + k
    , j*cell_w + l, 2]
     return output.reshape(-1)
```

2.2.5 Implémentation HSV-HOG

Nous avons utilisé une représentation HSV (`compute_hog_hsv`), qui sépare la teinte (*hue*). Nous extrayons les cannaux H (hue) et V (value) qui caractérisent réciproquement la teinte et l'intensité du pixel. Puis, le calcul est simplement celui de HOG classique mais sur les deux matrices extraites. Le résultat est ensuite concaténé en un seul vecteur. Cette représentation est plus robuste aux variations d'illumination que RGB. La grille est 4×4 avec $B = 8$ bins.

```
def compute_hog_hsv(image, nb_height_cells=4, nb_width_cells=4,
    nb_bins=8):
        img_hsv = color.rgb2hsv(image)
        img_h=img_hsv[:,:,0]
        img_v=img_hsv[:,:,2]

        h_edge_h = sobel_h(img_h)
        v_edge_h = sobel_v(img_h)
        h_edge_v = sobel_h(img_v)
        v_edge_v = sobel_v(img_v)
        magnitude_h = np.sqrt(h_edge_h**2 + v_edge_h**2)
        magnitude_v = np.sqrt(h_edge_v**2 + v_edge_v**2)

        orientation_h = np.arctan2(v_edge_h, h_edge_h)
        orientation_v = np.arctan2(v_edge_v, h_edge_v)
        orientation_h[orientation_h < 0] += np.pi
        orientation_v[orientation_v < 0] += np.pi

       [...]

        for i in range(nb_height_cells):
            for j in range(nb_width_cells):
                for k in range(cell_h) :
                    for l in range(cell_w) :
                        angle_h = orientation_h[i*cell_h+k,j*cell_w+l]
                        angle_v = orientation_v[i*cell_h+k,j*cell_w+l]
```

```
                        bins_ind_h=int(angle_h//bin_width)
                        bins_ind_v=int(angle_v//bin_width)
                        if bins_ind_h==nb_bins :
                            bins_ind_h-=1

                        if bins_ind_v==nb_bins :
                                bins_ind_v-=1

                        output_h[i,j,bins_ind_h] += magnitude_h[i*
cell_h+k,j*cell_w+l]
                        output_v[i,j,bins_ind_v] += magnitude_v[i*
cell_h+k,j*cell_w+l]

     output = np.concatenate((output_h.reshape(-1),output_v.
reshape(-1)))
     return output.reshape(-1)
```

2.3 Réduction dimensionnelle : PCA par classe

2.4 Calcul de la PCA : Deux approches mathématiques

L'Analyse en Composantes Principales (PCA) se calcule via la matrice de covariance empirique. Soit $\mathbf{X}_{centered} \in \mathbb{R}^{n \times d}$ la matrice des données centrées, avec n échantillons et d variables (ex : les pixels). La matrice de covariance $\mathbf{C}$ est donnée par :

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}_{centered}^T \mathbf{X}_{centered} \in \mathbb{R}^{d \times d} \tag{8}$$

La décomposition en valeurs propres de cette matrice s'écrit :

$$\mathbf{C} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T \tag{9}$$

Où :

- $\mathbf{V}$ est une matrice orthogonale dont les colonnes sont les vecteurs propres (les directions principales).
- $\mathbf{\Lambda}$ est une matrice diagonale contenant les valeurs propres λ_i, représentant la variance associée à chaque axe.

Pour projeter les données sur les k premières composantes, on applique $\mathbf{Z} = \mathbf{X}_{centered} \mathbf{V}_k$. Cependant, la complexité algorithmique de cette décomposition est de $\mathcal{O}(d^3)$, ce qui s'avère prohibitif en traitement d'image où le nombre de dimensions d est particulièrement élevé.

Pour contourner ce goulot d'étranglement, les bibliothèques d'apprentissage automatique utilisent la Décomposition en Valeurs Singulières (SVD) directement sur $\mathbf{X}_{centered}$:

$$\mathbf{X}_{centered} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \tag{10}$$

Où :

- $\mathbf{U}$ contient les vecteurs singuliers gauches (liés à l'espace des échantillons).

— $\mathbf{\Sigma}$ est une matrice diagonale contenant les valeurs singulières σ_i ordonnées de façon décroissante.
— $\mathbf{V}$ contient les vecteurs singuliers, qui correspondent exactement aux vecteurs propres (ou directions principales) de la PCA cherchés précédemment.

Le lien mathématique direct avec la matrice de covariance s'exprime ainsi :

$$\mathbf{C} = \frac{1}{n-1}\mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T \tag{11}$$

On en déduit que les valeurs propres (variances) se calculent analytiquement à partir des valeurs singulières : $\lambda_i = \frac{\sigma_i^2}{n-1}$. La nouvelle projection se simplifie en $\mathbf{Z} = \mathbf{U}\mathbf{\Sigma}$. Cette méthode abaisse drastiquement la complexité à $\mathcal{O}(ndk)$ pour extraire les k composantes principales, ce qui la rend indispensable pour les bases de données à haute dimensionnalité.

Nous avons implémenté cette méthode avec la librairie sklearn :

```
class PCAInfoPreprocessing(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        self.pca_per_class = []
        for race in np.unique(y):
            pca = PCA(n_components=self.n_components)
            pca.fit(X[y == race])
            self.pca_per_class.append(pca)
        return self

    def transform(self, X):
        return np.hstack([p.transform(X) for p in self.pca_per_class
   ])
```

Listing 2 – Transformateur PCA par classe (extrait commun aux deux scripts).

2.5 Analyse de la variance cumulée et sélection des composantes

Pour évaluer la répartition de l'information au sein de notre espace de représentation initial, nous avons étudié la variance expliquée cumulée par les axes propres de la PCA globale. La Figure 3 illustre cette distribution spectrale.

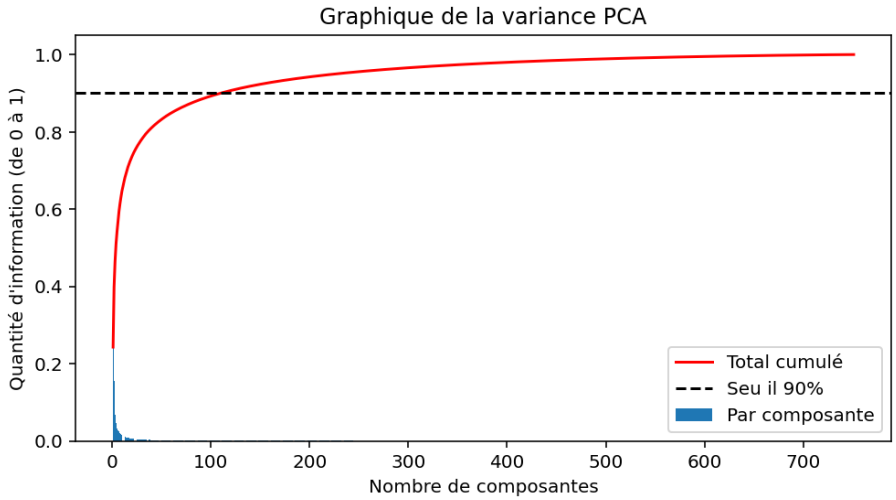


FIGURE 3 – Graphique de la variance PCA globale. Les barres bleues représentent la variance propre de chaque composante, la courbe rouge affiche le total cumulé et la ligne pointillée noire matérialise le seuil de rétention de 90 % de l'information.

L'examen du spectre de variance met en évidence la grande complexité de la base de données `SmallDB`. En effet, il est nécessaire de retenir environ **100 vecteurs principaux** pour atteindre le seuil de **90 %** de la variance totale cumulée. En deçà de ce nombre, une part non négligeable de l'information structurelle des images est purement tronquée.

Cette observation mathématique permet d'analyser de manière critique les grilles d'hyperparamètres explorées par la suite (Tableau 1). Alors qu'une projection globale exige une centaine de dimensions pour être fidèle, nos pipelines se restreignent volontairement à un nombre de composantes compris entre 5 et 20. Cette forte réduction est rendue possible par deux facteurs :

— **L'efficacité de la PCA par classe :** En ajustant un modèle par race de chien, l'espace d'apprentissage est géométriquement plus homogène, ce qui permet de capter les variations morphologiques majeures avec beaucoup moins d'axes propres que lors d'une décomposition globale.

— **La complémentarité du modèle « Lego » :** La perte d'information induite par la forte troncature de la PCA ($n \leq 20$) est avantageusement compensée par le descripteur HOG. Ce dernier prend le relais sur l'extraction des variations locales à haute fréquence (contours, textures de pelage), préservant le pouvoir discriminant global du classifieur.

2.6 Pipeline complet

Pour chacune des methodes, on construit un pipeline de la forme :

`MinMaxScaler` → `FeatureUnion(PCA + HOG/Couleur)` → `StandardScaler` → `SVC`

Le `MinMaxScaler` normalise d'abord les pixels entre $[0, 1]$. Le `FeatureUnion` fusionne horizontalement les descripteurs PCA et HOG. Le `StandardScaler` centre et réduit l'espace de caractéristiques avant l'apprentissage SVM.

2.7 Optimisation des hyperparamètres (GridSearchCV)

Un `GridSearchCV` à 3 folds explore l'espace des hyperparamètres suivant :

TABLE 1 – Grilles de recherche des hyperparamètres pour les deux équipes.

Hyperparamètre	**Raph**	**Orion**
`n_components` (PCA)	[5, 10, 20]	[5, 10, 15, 20]
`C` (SVM)	[0.1, 1, 10, 100]	[0.1, 10, 100]
`gamma` (SVM)	[`'scale'`, 0.01, 0.1]	[`'scale'`, 0.001]

TABLE 2 – Meilleurs scores obtenus par GridSearchCV (kernel RBF, 3-fold CV).

Approche	**Meilleurs paramètres**	**Score test (%)**
BW (baseline)	C=10,$\gamma = 0.01$n=5	60.16
HSV	C=100, γ=scale, n=15	**61.35**
RGB	C=100, γ=scale, n=20	51.00

Les meilleurs paramètres identifiés par les deux équipes convergent vers `C=100`, `gamma='scale'`, avec respectivement `n_components=20` (RGB, score 51 %) et `n_components=15` (HSB, score 61,35 %). On remarque que la méthode RGB a besoins de plus de vecteurs principaux que la méthode HSV et obtient quand même un score de généralisation inférieures a celle-ci. On note donc que la méthode HSV est plus efficace et plus précise. On explique cela car le canal RGB est sensible au variation de couleur du à la luminosité. Alors que le canal HSV sépare la luminosité et les motifs de couleur (H). Ce dernier canal (H) est optimale pour que le HOG trouve des contours nets et fiables.

3 OvO vs OvR

3.1 Contexte : le problème multiclasse des SVM

Les SVM sont des classifieurs binaires. Pour traiter un problème à $K = 6$ classes, deux stratégies de décomposition sont classiquement utilisées. Nous utiliserons les hyperparametres sélectionnés par le gridsearch à partir de maintenant.

3.1.1 One-vs-One (OvO)

OvO entraîne $\binom{K}{2} = \frac{K(K-1)}{2}$ classifieurs binaires, un pour chaque paire de classes. Pour $K = 6$, cela représente **15 classifieurs**. La prédiction finale est obtenue par vote majoritaire. OvO est préféré lorsque les données sont déséquilibrées.

3.1.2 One-vs-Rest (OvR)

OvR entraîne K classifieurs, chacun opposant une classe à toutes les autres. Pour $K = 6$, cela représente **6 classifieurs**. La classe prédite est celle dont le score de confiance est le plus élevé. OvR est plus rapide à entraîner mais peut souffrir d'un déséquilibre artificiel entre classes positives et négatives.

3.2 Comparaison des temps d'entraînement

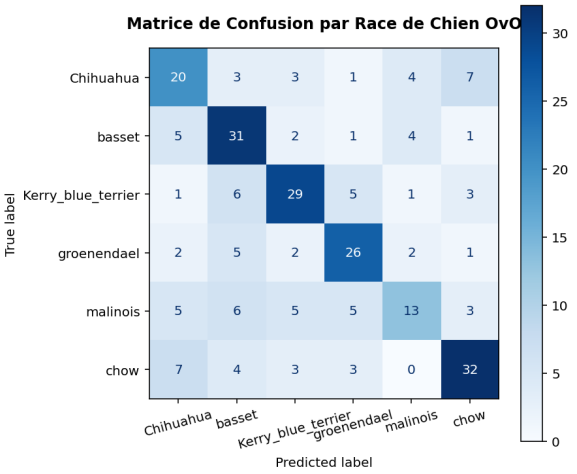
TABLE 3 – Temps d'entraînement et scores OvO/OvR pour les deux équipes.

protocole	Stratégie	Temps (s)	Accuracy (%)
BW	OvO	13.8	60.16
	OvR	15.5	57.37
HSV	OvO	20.48	60.16
	OvR	20.76	62.15
RGB	OvO	25.74	51.39
	OvR	27.58	49.80

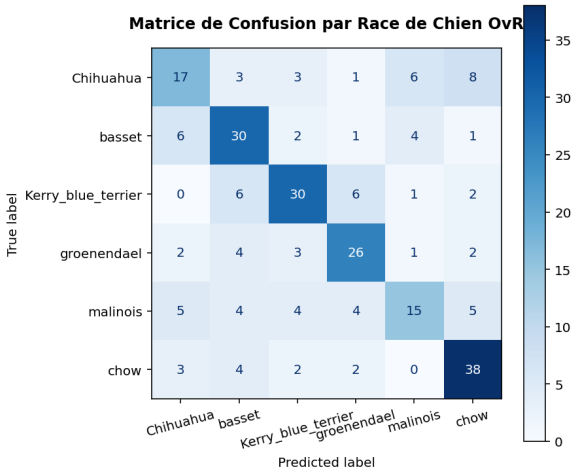
On observe que les temps d'entraînement sont comparables entre OvO et OvR pour $K = 6$, OvR étant légèrement plus lent dans tout les cas (davantage de données par classifieur). Les précisions entre Ovo et OvR sont aussi similaires dans tout les cas, OvR étant très légèrement meilleur. En résumé OvO est un peu plus rapide et OvR un peu plus précis.

RGB est le plus mauvais des modèles avec une précision de 50.60 au mieux. On explique ceci car les matrices de RGB sont très sensible au variation de couleur du à la luminosité ce qui pollue la reconnaissance des contours. BW est meilleur que RGB et le modèle le plus rapide de tous, en effet c'est le modèle qui à le moins de calcul à effectué. Même si il a moins de parmètre que RGB, BW est meilleur car L'algorithme est "forcé" de se concentrer uniquement sur les formes, les textures et les contrastes du chien. HSV fournit la meilleur précision, légérement mieux que BW en prenant plus d'information avec une information qui se concentre sur les formes et les contrastes. Il est cependant un peu plus long.

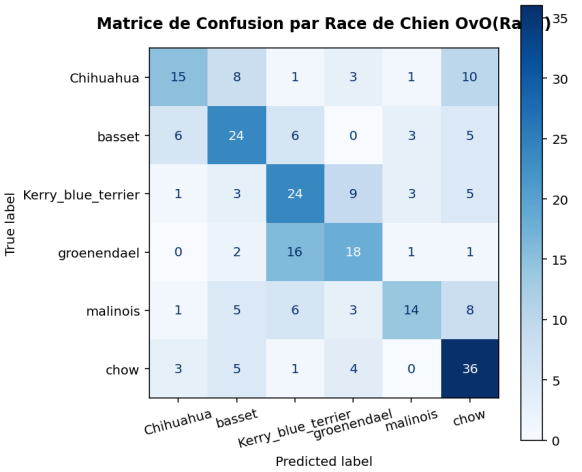
3.3 Matrices de confusion



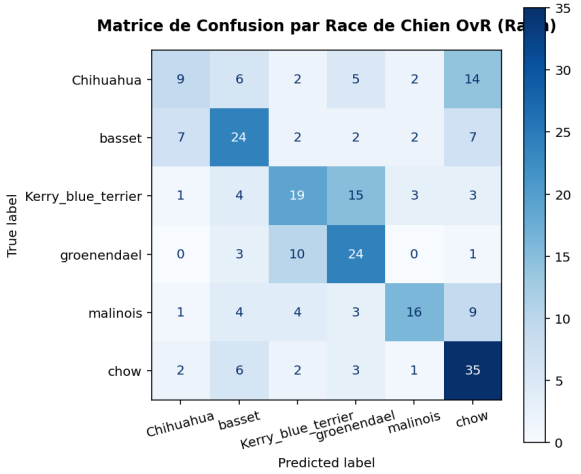
(a) OvO – HSV (60.16 %)



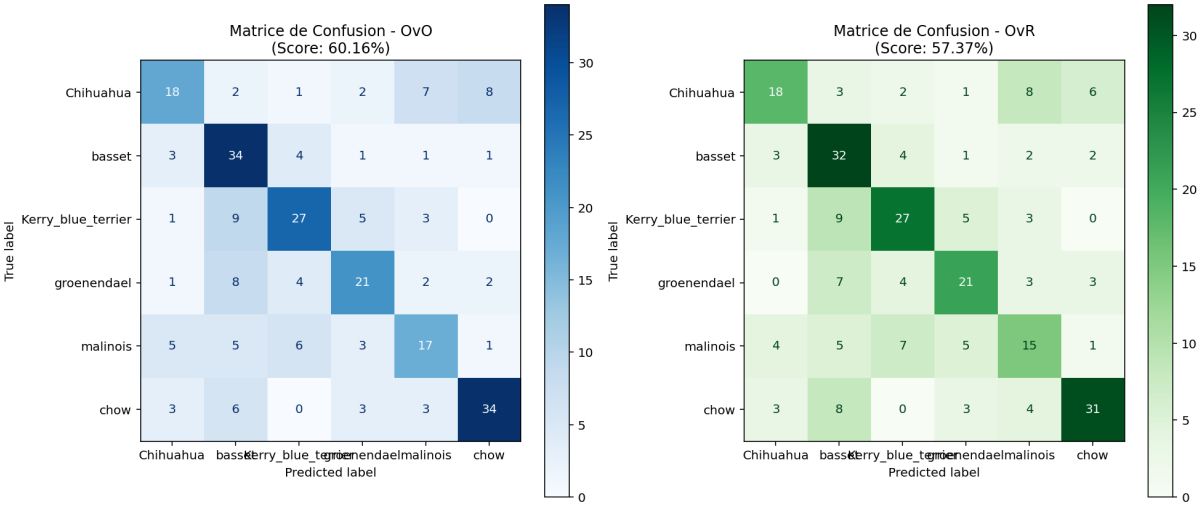
(b) OvR – HSV (62.15 %)



(a) OvO – RGB (52.19 %)



(b) OvR – RGB (50.60 %)



(a) OvO – BW (60.16 %) OvR – BW (57.37 %

3.4 Analyse des erreurs

Les matrices de confusion révèlent plusieurs tendances communes aux deux pipelines :

- Chow est la classe la mieux reconnue ($\geq$33/49 dans tous les modèles), probablement grâce à sa fourrure volumineuse et sa couleur caractéristique.
- Groenendael et Kerry Blue Terrier sont systématiquement confondus (7–16 erreurs croisées), ces deux races de bergers ayant des morphologies similaires.
- Malinois présente le taux de reconnaissance le plus faible, souvent confondu avec Basset et Kerry Blue Terrier.
- Chihuahua souffre de nombreuses confusions avec Chow, ce qui suggère une sensibilité aux caractéristiques de couleur plutôt que de forme.

3.5 Discussion

Apport de la couleur. L'approche HSV surpasse la celle en niveaux de gris et l'approche RGB. L'information chromatique est discriminante pour distinguer des races comme

le Chow (fourrure fauve) du Groenendael (fourrure noire). HSV est meilleure que RGB car celle-ci considere les varaitions de luminosité et de teinte en meme temps tandis que HSV les sépare (par exemple les ombrages peuvent tromper la méthode RGB)

Résolution et padding. Le padding continus a l'avantage de ne pas induire des varaitions brutales sur l'image ce qui pourrait tromper HOG. C'est pourquoi nous avons preferé le padding continu au noir et au blanc.

Limites communes. Les deux approches peinent sur les races morphologiquement proches (Groenendael/Kerry Blue Terrier, Malinois). Des pistes d'amélioration incluent : l'utilisation de descripteurs plus riches (LBP, SIFT), l'augmentation de données, ou le recours à des réseaux de neurones pour l'extraction automatique de caractéristiques.

4 Application à la Reconnaissance Faciale

4.1 Protocole expérimental et base de données

Contrairement à l'étude précédente portant sur la classification de races de chiens, cette section explore l'application de l'Analyse en Composantes Principales (PCA) et des classifieurs géométriques sur des visages humains. Pour cette expérience, une base de données personnalisée a été constituée à partir d'une vingtaine de photographies pour chacun des 9 membres de l'équipe (*Anir, Evrard, Florent, James, Matteo, Nathan, Orion, Raphael* et *Yassine*).

Toutes les images ont subi une étape rigoureuse de prétraitement : conversion en niveaux de gris, redimensionnement à une résolution standardisée de 112×92 pixels et normalisation de l'intensité des pixels. Les données ont ensuite été partitionnées de manière stratifiée en un ensemble d'entraînement (80 %) et un ensemble de test (20 %). Le pipeline de prédiction combine une réduction de dimension par PCA (préservant les 15 premières composantes) et un classifieur basé sur la règle du plus proche voisin (*1-NN*).

4.2 Performances de classification et analyse de la séparabilité

Le modèle de reconnaissance faciale ainsi configuré atteint une exactitude (*accuracy*) parfaite de **100 %** sur l'ensemble de test. Comme en témoigne la matrice de confusion (Figure 7), l'intégralité des visages de test a été correctement affectée à son identité respective, sans aucune confusion inter-classe.

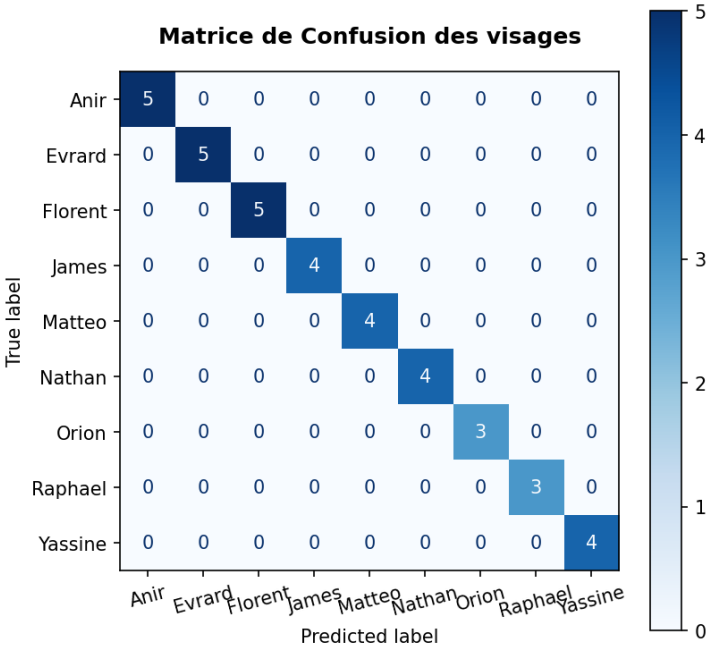


FIGURE 7 – Matrice de confusion du modèle de reconnaissance faciale (PCA + 1-NN). Le modèle obtient un score parfait de 100 % sur l'ensemble des 9 classes de visages.

Ce contraste saisissant avec les performances obtenues sur les races de chiens (qui plafonnaient autour de 61 %) s'explique par deux phénomènes majeurs :

— **La haute standardisation du jeu de données :** Les visages ont été capturés sous des angles de prise de vue similaires (face stricte), avec un arrière-plan neutre et des conditions d'éclairage maîtrisées. À l'inverse, la base `SmallDB` de chiens intègre de fortes variations de postures, de cadrages et d'environnements complexes.

— **La sensibilité géométrique de la PCA :** Souvent qualifiée d'approche par *« Eigenfaces »*, la PCA est extrêmement sensible à l'alignement des pixels. Lorsque les structures anatomiques (yeux, nez, bouche) coïncident spatialement d'une image à l'autre, la PCA capture de manière optimale les variations morphologiques individuelles, rendant les projections dans l'espace latent parfaitement séparables pour le classifieur 1-NN.

4.3 Reconstruction d'image et approximation spectrale

Pour analyser la capacité de la PCA à compresser et à restituer l'information visuelle d'un visage, nous avons étudié la reconstruction de l'image de James en faisant varier le nombre de composantes principales retenues, noté k, de 1 à 15 (Figure 8).



FIGURE 8 – Reconstruction du visage de James par projection inverse PCA en fonction du nombre de composantes principales k, accompagnée de l'évolution de l'erreur quadratique L_2.

L'examen de la Figure 8 met en évidence les propriétés de convergence de la décomposition :

— **Dès la première composante ($k = 1$)** : Bien que l'image soit très floue et présente l'erreur quadratique la plus élevée ($L_2 = 6,41$), les structures fondamentales du visage (le contour global, la position des yeux et l'implantation de la barbe) sont déjà instantanément reconnaissables. Cela confirme que le premier axe propre capture les variations macroscopiques et la signature géométrique principale de l'individu.

— **Comportement asymptotique de l'erreur** : L'erreur L_2 chute de manière drastique au passage des premières composantes, glissant à $3,48$ pour $k = 2$ et $2,32$

pour $k = 3$. Elle se stabilise ensuite sous la barre des $0,30$ dès que $k \geq 11$.

— **Haute fidélité à $k = 15$** : Avec 15 composantes, l'erreur résiduelle n'est plus que de $0,09$. Les détails haute fréquence (texture fine de la peau, reflets dans les yeux, ombrages précis) sont fidèlement restitués. L'image reconstruite est visuellement indiscernable de l'originale, illustrant l'efficacité de la troncature SVD pour la compression d'images alignées.

4.4 Génération de visages synthétiques et visage chimère

En exploitant les propriétés de l'espace latent de la PCA, la fonction `generer_visage_aleatoire` permet de synthétiser des visages inédits. En effectuant un tirage aléatoire gaussien calibré par les variances apprises ($\sigma_i = \sqrt{\lambda_i}$), on génère des coordonnées latentes réalistes qui, après transformation inverse, produisent un visage plausible propre à une identité.

Pour conclure cette étude sur une note plus récréative, le modèle a été entraîné sur la totalité des visages combinés de la classe afin de générer un **visage chimère** (Figure 9). En échantillonnant au hasard au sein de cet espace propre global, le modèle synthétise une image qui fusionne les caractéristiques latentes partagées de James, Orion, Raphael et de l'ensemble de leurs camarades. Ce portrait-robot hybride matérialise de façon concrète et ludique le concept d'« espace de visages » commun sous-tendu par la théorie des Eigenfaces.



FIGURE 9 – Visage « chimère » synthétique obtenu par un échantillonnage aléatoire au sein de l'espace latent global, combinant les traits géométriques et morphologiques de l'ensemble des étudiants.

Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de classification d'images : prétraitement, extraction de caractéristiques (HOG, couleur, PCA par classe), optimisation d'hyperparamètres et comparaison de stratégies multiclasses. L'approche d'Orion, basée sur un HOG en espace HSV avec une résolution 128×128, atteint la meilleure précision (61,35 %) parmi les configurations testées. Les principales sources d'erreurs sont les confusions entre races morphologiquement similaires, problème que des architectures plus profondes (CNN) pourraient atténuer.